

CONTENTS

- Overview
- 1 Keyword search: inverted indices and ranking
- 2 Where lexical matching breaks
- 3 Semantic search: meaning as geometry
- 4 Hybrid retrieval: precision and recall together
- 5 LLMs predict — they do not retrieve
- 6 Naive RAG: the linear pipeline
- 7 Advanced RAG: rerankers, query rewriting, hybrid
- 8 Agentic RAG: retrieval as a tool inside reasoning
- Glossary
- Check yourself
- Where to go next

From Inverted Indices to Agentic RAG: How Retrieval Grew Up

Follow information retrieval from keyword matching to systems that decide for themselves what to look at.

For engineers building search or RAG systems who want to know which stage of the evolution their system is stuck at · 26 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Familiarity with the idea of a search index
- Comfort reading Python
- A basic idea of what an embedding is, though the lesson builds it up

Overview

Search engines did not get smarter overnight. Information retrieval grew up one step at a time, and each step was a response to a specific, nameable failure of the step before it. Keyword indices could not handle synonyms. Semantic search could not handle exact identifiers. Static RAG pipelines could not adapt when one retrieval was not enough.

That progression is worth knowing in detail, because almost every retrieval system in production today is sitting at one of these stages, and its problems are the documented problems of that stage. A team fighting recall issues with better embeddings is often at the point where hybrid retrieval was invented. A team whose RAG works on simple questions and collapses on multi-hop ones is at the point where agentic retrieval was invented.

The endpoint so far is retrieval as a tool inside a reasoning loop, rather than a fixed step ahead of one. The agent decides whether to search, where, what to ask, and when it has enough. As the video puts it: the hardest part is not generation, it is deciding what to look at.

KEY TAKEAWAYS

- ✓ An inverted index maps terms to the documents containing them, which is what makes keyword search fast.
- ✓ BM25 ranks by term frequency, inverse document frequency and document length, and remains a strong baseline decades on.
- ✓ Lexical search treats words as symbols, so synonyms, ambiguity and intent are invisible to it.
- ✓ Embeddings are learned from context, which is why 'espresso' lands near 'coffee' and far from 'house'.
- ✓ Hybrid retrieval combines lexical precision with semantic recall because each covers the other's failures.
- ✓ LLMs predict likely tokens rather than retrieving facts, so their knowledge is frozen and unattributable.
- ✓ Naïve RAG is linear — embed offline, retrieve once, generate — and is only ever as good as that single retrieval.
- ✓ Agentic RAG makes retrieval a tool invoked during reasoning, so the system can iterate, verify and stop when it has enough.

01 Keyword search: inverted indices and ranking

0:00

The earliest systems answered one question — where does this word appear — using a mapping from terms to documents, then ranked the hits by statistical importance.

The earliest search systems were designed around the question 'where does this word appear?'. Documents were indexed using inverted indices: a mapping from keywords to the documents that contain them. It is called inverted because the natural direction is document to words, and the index stores the reverse. When a user asks a question, the system looks up each term and quickly returns the matching documents. The cost is proportional to the number of documents containing the query terms rather than to the size of the corpus, which is what makes it fast at web scale.

Matching alone gives you a set, not an order, so the results are ranked by measures of how important or frequent the terms are. TF-IDF was the classic: term frequency, how often the term appears in this document, multiplied by inverse document frequency, which discounts terms appearing everywhere. A word occurring in every document tells you nothing and gets weighted to near zero.

BM25 refined this and remains the standard baseline. It adds two corrections that matter in practice. Term frequency saturates, so the tenth occurrence of a word counts far less than the second — a document repeating a term fifty times is not fifty times more relevant. And it normalises by document length, so a short document containing your term is not out-ranked by a long one that merely happened to contain it too.

This approach still powers a great deal of the internet, and it should not be filed under history. BM25 is fast, needs no model, requires no training, is completely interpretable, and beats a badly tuned vector search more often than vector-search enthusiasts expect.

KEY POINTS

- An inverted index maps each term to the documents containing it, making lookup proportional to matches rather than corpus size.
- TF-IDF weights a term by its frequency in the document against its rarity across the corpus.
- BM25 adds term-frequency saturation so repetition has diminishing returns.
- BM25 also normalises for document length, so short focused documents are not penalised.
- Lexical ranking needs no model or training and is fully interpretable — it remains a serious baseline.

An inverted index, in full

The whole data structure. Querying 'python error' intersects two posting lists and returns their overlap. Real implementations add positions for phrase queries and skip lists for faster intersection, but this is the idea in its entirety.

```
from collections import defaultdict

index = defaultdict(set)

def add(doc_id, text):
    for term in tokenize(text.lower()):
        index[term].add(doc_id)

add(1, "Python error handling with try and except")
add(2, "Ball python care and feeding")
add(3, "Error codes in the payment API")

# index["python"] = {1, 2}
# index["error"] = {1, 3}

def search(query):
    sets = [index[t] for t in tokenize(query.lower()) if t in index]
    return set.intersection(*sets) if sets else set()

search("python error")  # {1}
```

BM25, term by term

The formula that has held up for thirty years. `k1` controls how quickly term frequency saturates and `b` controls how strongly length normalisation applies; the defaults below are the usual starting point and are rarely worth changing much.

```
import math

def bm25(query_terms, doc, corpus_stats, k1=1.5, b=0.75):
    N      = corpus_stats.num_docs
    avgdl  = corpus_stats.avg_doc_len
    dl     = len(doc.tokens)
    score  = 0.0

    for term in query_terms:
        n_q = corpus_stats.doc_freq[term]      # docs containing the term
        if n_q == 0:
            continue

        idf = math.log((N - n_q + 0.5) / (n_q + 0.5) + 1)  # rare terms weigh more
        f   = doc.term_freq[term]

        # saturating term frequency, normalised by document length
        score += idf * (f * (k1 + 1)) / (f + k1 * (1 - b + b * dl / avgdl))

    return score
```

02 Where lexical matching breaks

1:24

It treats words as symbols, not as meaning. Synonyms, ambiguity and intent are invisible.

There is a fundamental limitation to keyword matching: it does not understand language. It treats words as symbols. Two strings either match or they do not, and everything that makes language work — that different words can mean the same thing, that the same word can mean different things — is outside the model entirely.

Synonymy is the recall failure. A user searching for 'car insurance' gets nothing from a document that consistently says 'vehicle coverage', even though it is exactly what they wanted. The information is there and the index cannot connect the two.

Ambiguity is the precision failure. Is the search 'help Python' about coding, or did somebody just get a pet snake? The tokens are identical in both cases. A lexical index has no basis for choosing, so it returns both and lets the user sort it out.

The burden therefore falls on the user to ask with the exact right words — to guess the vocabulary the author happened to use. That is a strange thing to require, and it is the specific problem the next stage set out to fix.

KEY POINTS

- Lexical matching compares symbols, so it has no access to meaning.
- Synonymy causes recall failures: the right document exists and cannot be found.
- Ambiguity causes precision failures: identical tokens, different intents.
- The user is forced to guess the author's exact vocabulary.
- Neither failure can be fixed by better ranking — the candidates were never retrieved.

Two failures, two causes

Worth separating, because the fixes differ. Synonym failures are about candidate generation and need semantics. Ambiguity failures are about disambiguation and often need context about the user or the session.

SYNONYMY (recall)

```
query : "car insurance"
doc   : "Comprehensive vehicle coverage for private owners"
match : none. zero shared terms, and the document is the right answer.
```

AMBIGUITY (precision)

```
query : "help python"
doc A : "Debugging Python tracebacks"      ← wanted, if a programmer
doc B : "Ball python humidity requirements" ← wanted, if a snake owner
match : both, ranked by term statistics alone.
```

The workarounds, and why they ran out

These were the pre-semantic answers, and they are still useful — but each one requires someone to write and maintain a list, which does not scale to open-domain language or keep up with new terminology.

stemming	"running" → "run"	mechanical, cheap, limited
synonym lists	car ≠ vehicle ≠ automobile	hand-curated, never complete
query expansion	add related terms manually	improves recall, hurts precision

all of them are attempts to approximate meaning with string rules

03 Semantic search: meaning as geometry

2:09

Represent text as vectors in a high-dimensional space where similar concepts sit close together, and relevance becomes distance.

The next major leap was to stop treating text as words and start representing it as meaning. This is done with vectors: high-dimensional numeric representations positioned so that related concepts land near one another. In a toy three-dimensional space, coffee might be $[0, 1, 0]$ while house is $[1, 0, 0]$ — different directions entirely.

These embeddings do not come from nowhere. They are learned by large neural networks trained on massive text corpora. By repeatedly encountering words in context, the network arranges the space so that concepts appearing in similar contexts end up close together, even when the words themselves share no letters. Espresso lands near coffee — very close in concept — and nowhere near house. The insight the whole field rests on is that meaning can be inferred from company: words that appear in similar surroundings mean similar things.

Semantic search turns your query into a point on that map, so the system knows espresso and coffee point to a very similar place. It is the friend who knows what you meant even when you did not say it perfectly.

This let search systems approximate intent. Even without the exact keywords, you could find relevant documents — the synonymy problem dissolves, because 'vehicle coverage' and 'car insurance' occupy nearly the same region regardless of sharing no characters.

KEY POINTS

- Embeddings are vectors positioned so that semantic similarity becomes geometric proximity.
- They are learned by neural networks from context across massive corpora.
- Words in similar contexts end up in similar positions, independent of spelling.
- Retrieval becomes a nearest-neighbour problem rather than a matching problem.
- Synonymy is solved structurally: different words for one concept share a region.

The map, in three dimensions

Real embeddings have hundreds or thousands of dimensions and no individual axis means anything interpretable. But the geometry is genuinely this: similarity is a small angle, and unrelatedness is a large one.

```
coffee    [0.0, 1.0, 0.0]
espresso   [0.1, 0.9, 0.1]    ← near coffee: same contexts, same region
latte      [0.1, 0.8, 0.2]
house      [1.0, 0.0, 0.0]    ← far: nothing in common

cosine(coffee, espresso) ≈ 0.99
cosine(coffee, house)    ≈ 0.00

# no shared letters between "coffee" and "espresso" – the closeness comes
# entirely from having been used in similar surroundings during training
```

Semantic retrieval end to end

Note where the work happens. Documents are embedded once, offline. At query time you embed one short string and do a nearest-neighbour search — which is why this is fast despite involving a neural network.

```
# offline, once per document
vectors = {doc.id: embed(doc.text) for doc in corpus}

# at query time
def semantic_search(query, k=5):
    q = embed(query)
    ranked = sorted(vectors.items(), key=lambda kv: -cosine(q, kv[1]))
    return [doc_id for doc_id, _ in ranked[:k]]

semantic_search("car insurance")
# returns the "comprehensive vehicle coverage" document, which shares no terms
```


What semantics loses

The trade nobody mentions when introducing embeddings. Vectors are excellent at concepts and poor at exact tokens, because an identifier's meaning is not distributed across contexts — it is arbitrary. This is the failure that motivates the next section.

```
query: "error TS2345"
```

```
lexical : exact match, one document, correct
```

```
semantic : returns documents about "TypeScript type errors" generally,  
           and may rank the exact match below them
```

```
# the same for SKUs, error codes, function names, version numbers,
```

```
# legal citations, part numbers – anything where the string is the meaning
```

04 Hybrid retrieval: precision and recall together

2:49

Semantic search did not replace keyword search. Systems combining both emerged because each covers the other's failures exactly.

Semantic search allowed systems to understand intent, and to find relevant documents even without the exact keywords. But it did not replace keyword search — it complemented it. Hybrid systems emerged, bridging the precision of keyword matching with the recall of semantic search. For the first time, instead of just matching text, search could approximate understanding.

The reason hybrid works so well is that the two methods fail in opposite directions. Lexical search is precise and brittle: it nails exact identifiers, error codes and product numbers, and misses paraphrase entirely. Semantic search is fuzzy and robust: it handles paraphrase and intent, and can rank an exact-match document below a topically similar one. Union the candidates and you have covered both classes.

The practical difficulty is combining scores. BM25 scores are unbounded and corpus-dependent; cosine similarities sit in a fixed range. Normalising them into comparability is fiddly and fragile. Reciprocal Rank Fusion sidesteps the problem entirely by discarding the scores and combining ranks, which is why it has become the default: it needs no tuning, no normalisation, and no knowledge of either scoring scale.

KEY POINTS

- Hybrid retrieval unions lexical and semantic candidates rather than choosing between them.
- Lexical is precise on exact tokens; semantic is robust to paraphrase.
- The two fail in opposite directions, so together they cover far more ground.
- Combining raw scores is fragile because the scales are incomparable.
- Reciprocal Rank Fusion combines ranks instead of scores and needs no tuning.

Reciprocal Rank Fusion

The whole algorithm. Each document scores the sum of $1/(k + \text{rank})$ across the lists it appears in, so appearing reasonably high in both beats topping one and missing the other. The constant k , conventionally 60, damps the influence of the very top positions.

```
def rrf(rankings, k=60):
    scores = defaultdict(float)

    for ranking in rankings:          # one list per retriever
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] += 1.0 / (k + rank)

    return sorted(scores, key=scores.get, reverse=True)

fused = rrf([
    bm25_search(query, k=50),          # exact terms, identifiers, codes
    semantic_search(query, k=50),      # paraphrase, intent
])

# no score normalisation, no weights to tune, no scale assumptions
```

Each retriever rescuing the other

Two real query shapes. Neither retriever handles both, and no amount of tuning makes one of them do so — which is the argument for running both.

query: "error TS2345"

bm25 → exact match at rank 1

semantic → exact match at rank 7, generic type-error docs above it

fused → exact match at rank 1 (lexical rescues semantic)

query: "why does my build fail after upgrading?"

bm25 → nothing useful; no distinctive terms to match on

semantic → the right troubleshooting guide at rank 1

fused → the right guide at rank 1 (semantic rescues lexical)

05 LLMs predict — they do not retrieve

3:31

A language model returns the most likely continuation based on learned patterns. It has no index, no sources and no notion of looking something up.

Then the world shifted. Large language models arrived — models trained on huge corpora to learn patterns in the data. Ask one a question and it returns a text answer, and the answers were good enough to change how businesses worked.

But the mechanism matters. LLMs do not retrieve facts. When prompted, they predict the most likely next token based on patterns learned during training. There is no lookup, no index, no source. What feels like recall is a very good statistical continuation.

Two consequences follow directly. Knowledge is locked to whatever documents the model was trained on, before a certain point in time — so it does not know today's information, and certainly does not know your specific documents. And because there was never a source, there is nothing to cite: a model asked for a reference will generate a plausible-looking one with exactly the same machinery it used for the answer.

So what fixes this? Search. The field had spent decades building systems that find relevant documents, and the thing language models most needed was precisely that.

KEY POINTS

- LLMs predict likely continuations; they do not look anything up.
- There is no index and no source, so nothing can be genuinely cited.
- Knowledge is frozen at the training cutoff and excludes anything private.
- Fluency is not evidence of grounding — the same mechanism produces both.
- The missing capability is exactly what information retrieval already provided.

Why a model invents citations

Not a special failure mode. The reference is generated by the same next-token prediction as the surrounding prose, and a plausible-looking citation is a highly probable continuation after the words 'according to'. Nothing in the process consults anything.

```
prompt : "According to the documentation, the retry limit defaults to"
```

```
# " 3" 0.41 plausible
```

```
# " 5" 0.28 plausible
```

```
# " 10" 0.12 plausible
```

```
# the model emits one. no configuration file was consulted, and the
```

```
# confidence of the output says nothing about whether it is right.
```

06 Naive RAG: the linear pipeline

4:20

Search for relevant documents, augment the prompt with them, generate the answer.
Simple, effective, and only as good as one retrieval.

Retrieval Augmented Generation was born, and the idea is very simple. The user asks a question. The system searches an external knowledge base for relevant documents. Those documents augment the LLM's prompt. A final answer is generated from them.

This gave language models a form of external memory. They could now cite sources, because there were real sources in the prompt to cite. They could adapt to new information as soon as it was indexed. And they could operate in specialised domains without any costly retraining. The drop in hallucinations was substantial, and it is what made LLMs viable across a wide range of applications that had been closed to them.

These original pipelines were strictly linear. Documents were embedded offline into vector databases. At query time, retrieval ran once, and the results passed straight into the model. Simple, but effective.

The limitation is contained in that word 'once'. Traditional RAG cannot adapt to new scenarios, and we are back at the old problem of search: the answer is only ever as good as the search itself. One shot, no feedback, no second attempt. If the first retrieval was poor, the pipeline has no mechanism to notice or to try again.

KEY POINTS

- The pipeline is: retrieve from a knowledge base, augment the prompt, generate.
- This gives the model external memory, real citations and domain adaptation without retraining.
- Hallucinations drop substantially because claims are grounded in supplied text.
- The pipeline is linear: retrieval happens exactly once, with no feedback.
- Answer quality is capped by the quality of that single retrieval.

Naive RAG in full

Genuinely the whole thing. Everything covered from here on is an attempt to fix a limitation of these five lines.

```
def naive_rag(question, store, k=5):
    hits = store.search(embed(question), k=k)      # once
    context = "\n\n".join(h.text for h in hits)

    return model.generate(
        f"Answer from the context only.\n\n{context}\n\nQuestion: {question}"
    )
```

The question shape it cannot handle

A multi-hop question. The second search depends on the answer to the first, and a single retrieval step has no way to run it. The pipeline retrieves for the literal question, finds nothing that matches, and produces a grounded-looking answer built from whatever came back.

```
question: "Does the vendor of our payment gateway have SOC 2 certification?"

hop 1 needed: which vendor supplies the payment gateway? → "Acme Pay"
hop 2 needed: is Acme Pay SOC 2 certified? → the actual answer

naive RAG: embeds the whole question, retrieves chunks mentioning
           "payment gateway" and "SOC 2" – which are different documents
           that never mention each other. no hop is ever performed.
```

Within a short period the simple concept accumulated real machinery — but the pipeline stayed fixed.

Countless advancements turned the simple concept into something considerably more powerful. Instead of a single retrieval step, pipelines added rerankers to reorder results by relevance. User queries were rewritten or expanded to improve recall. And hybrid retrieval became the norm, pairing lexical precision with semantic recall.

Rerankers are the highest-leverage of the three. Vector search uses a bi-encoder: query and document are embedded separately, which is what makes pre-computing the index possible, but it also means the two are never compared directly. A cross-encoder reads the query and the document together and scores them jointly, which is far more accurate and far too slow to run over a whole corpus. So you use both — retrieve fifty cheaply, rerank them expensively, keep five. Retrieve wide, then narrow.

Query rewriting attacks the other end. Users ask short, underspecified, conversational questions, and those embed poorly. Rewriting them into fuller standalone queries, or generating several variants and unioning the results, measurably improves recall.

These systems were far more accurate, but they remained fundamentally static. The pipeline was predetermined: the same steps in the same order for every query, regardless of whether the question was trivial or needed five hops. Retrieval had become smarter, but it was still not intelligent.

KEY POINTS

- Rerankers reorder a retrieved shortlist using a more accurate, slower model.
- Bi-encoders embed query and document separately; cross-encoders score them jointly.
- Retrieve wide and cheap, then rerank narrow and expensive.
- Query rewriting and expansion improve recall for short or conversational questions.
- The pipeline was still fixed — identical steps for every query, however simple or complex.

Retrieve wide, rerank narrow

Usually the single largest accuracy improvement available to a RAG system, and it needs no change to the index. The recall you get from k=50 with the precision you get from a cross-encoder.

```
def retrieve_and_rerank(question, store, wide=50, keep=5):
    candidates = store.search(embed(question), k=wide)      # cheap, high recall

    scored = cross_encoder.predict([                        # expensive, precise
        (question, c.text) for c in candidates
    ])

    ranked = sorted(zip(scored, candidates), reverse=True)
    return [c for _, c in ranked[:keep]]

# bi-encoder : embeds separately → index precomputable, less accurate
# cross-encoder: reads both together → far more accurate, cannot be precomputed
```

Query rewriting for conversational input

The follow-up question is meaningless on its own — 'it' and 'that' resolve only against the conversation. Embedding it retrieves nothing useful. Rewriting it into a standalone query is a small step that fixes a large class of failures in chat interfaces.

```
history : "Tell me about the retry policy." / "...exponential backoff, 5 attempts..."
follow-up: "Does it apply to webhooks too?"

# embedding the follow-up as written retrieves documents about the word "webhooks"
# and nothing about retries at all

rewritten = model.generate(
    f"Rewrite the follow-up as a standalone search query.\n\n"
    f"History:\n{history}\n\nFollow-up: {follow_up}"
)
# → "Does the exponential backoff retry policy apply to webhook deliveries?"
```

Multi-query expansion

One question, several phrasings, results fused. Different phrasings land in different regions of the embedding space, so the union recovers documents any single phrasing would have missed.

```
variants = model.generate(
    f"Write 3 alternative phrasings of this search query, one per line:\n{question}"
).splitlines()

results = [store.search(embed(v), k=20) for v in [question, *variants]]
final = rrf(results)[:10]
```

08 Agentic RAG: retrieval as a tool inside reasoning

6:29

Retrieval stops being a fixed step before the answer and becomes something the system invokes, repeats and stops as part of thinking.

Then agents arrived. Agents are systems that use LLMs and tools to perform tasks autonomously, and the shift is from simple pipelines to complex decision-making systems. An agent has a variety of tools available — retrievers, memory, planners, critics, other models — and it chooses among them.

Applied to retrieval, this changes the control flow completely. Instead of a linear retrieve-then-generate, the agent decides whether retrieval is needed at all, where to search, what questions to ask, when enough information has been gathered, and only then generates the final answer. It can compare sources, validate claims, refine queries and iterate. It can call APIs, pull from several knowledge bases and incorporate different modalities.

Retrieval is no longer fixed. It is a tool invoked as part of reasoning. That single change is what enables multi-step research, cross-document synthesis and adaptive behaviour — the system does not just answer questions, it works out how to answer them.

The cost is everything that comes with a loop. Latency becomes variable and data-dependent. Cost per query is no longer predictable. Evaluation gets harder, because you are grading a trajectory rather than a retrieval. And the system needs the usual defences: a cap on retrieval calls, a stopping criterion, and a rule for what to do when it has searched repeatedly and found nothing. An agent that cannot conclude the answer is absent will keep rephrasing the query until its budget runs out.

KEY POINTS

- Agents combine an LLM with tools and decide autonomously which to invoke.
- The agent decides whether to retrieve, where, what to ask, and when to stop.
- Retrieval becomes a tool inside reasoning rather than a fixed preceding step.
- This enables multi-hop research, cross-document synthesis and self-correction.
- Latency and cost become variable; evaluation shifts from one retrieval to a whole trajectory.
- Retrieval budgets and stopping criteria are required — an agent will otherwise search forever.

Agentic retrieval performing a multi-hop lookup

The same question naive RAG could not answer. The agent runs the first hop, reads the result, forms a second query that depends on it, and only then answers. No pipeline can do this, because the second query does not exist until the first one returns.

question: "Does the vendor of our payment gateway have SOC 2 certification?"

Thought: I need the vendor's name first. Search internal docs.

Action: `search_internal("payment gateway vendor contract")`

Obs: "Payments are processed via Acme Pay under MSA-2024-11."

Thought: Vendor is Acme Pay. Now check certification — that is external.

Action: `search_web("Acme Pay SOC 2 Type II report")`

Obs: "Acme Pay SOC 2 Type II, audited through 2025-06-30."

Thought: Both hops resolved, and the report has an end date worth flagging.

Answer: Yes — Acme Pay holds SOC 2 Type II covering through 30 June 2025.

Retrieval as one tool among several

Once retrieval is a tool, the agent can also decline to use it. A question answerable from the conversation should not trigger a search at all — and knowing when not to retrieve is a genuine efficiency gain over a pipeline that always does.

```
TOOLS = {
    "search_internal": "Search company wiki, runbooks and contracts.",
    "search_web":      "Search public sources. Use for external entities.",
    "read_document":   "Fetch a document in full by ID. Use when snippets are"
                      "insufficient to compare or count.",
    "answer":          "Give the final answer. Only when evidence is sufficient.",
}

# "what did I just ask you?"      → no retrieval. answer directly.
# "what is our retry policy?"    → one internal search.
# "which requirements were dropped?" → read_document twice, compare in full.
```

Bounding the loop

Without this an agent that cannot find something will rephrase indefinitely, because generating another query is always easier than concluding the information does not exist. Concluding absence has to be an explicitly permitted outcome.

```
MAX_RETRIEVALS = 6

def should_stop(state):
    if state.retrievals >= MAX_RETRIEVALS:
        return "budget exhausted – report what was found and what was not"
    if state.last_two_queries_returned_same_docs():
        return "no new information – stop searching"
    if state.critic_says_sufficient():
        return "evidence sufficient – answer now"
    return None

# "I searched and it is not there" must be a valid, rewarded outcome
```

Glossary

Inverted index

A mapping from each term to the documents containing it — the reverse of the natural document-to-terms direction, which is what makes lookup fast.

TF-IDF

Term frequency times inverse document frequency: a term matters more if it is frequent in this document and rare across the corpus.

BM25

The standard lexical ranking function, refining TF-IDF with saturating term frequency and document-length normalisation.

Lexical search

Retrieval by literal term matching. Precise on exact strings, blind to synonyms and intent.

Semantic search

Retrieval by proximity between embeddings, so documents match on meaning even when they share no words.

Embedding

A high-dimensional vector representing text, learned so that items appearing in similar contexts occupy similar positions.

Hybrid retrieval

Running lexical and semantic retrieval together and fusing the results, since each covers the other's characteristic failures.

Reciprocal Rank Fusion (RRF)

Combining several ranked lists by summing $1/(k + \text{rank})$ per document, avoiding the need to make incompatible scores comparable.

Bi-encoder

An embedding model that encodes query and document separately, allowing the corpus to be indexed in advance.

Cross-encoder

A model that reads query and document together and scores the pair. More accurate than a bi-encoder and too slow to run over a full corpus.

Reranker

A second-stage scorer, usually a cross-encoder, applied to a retrieved shortlist to reorder it more accurately.

Query rewriting

Reformulating a short or conversational question into a fuller standalone query before retrieval, to improve recall.

Naive RAG

The original linear pipeline: retrieve once, augment the prompt, generate. No feedback and no second attempt.

Agentic RAG

Retrieval invoked as a tool during reasoning, so the system decides whether, where and how often to search, and when it has enough.

Multi-hop question

A question whose answer requires a second lookup that depends on the result of the first. Beyond the reach of single-shot retrieval.

Check yourself

Answer first, then open each one.

Why does BM25 make the tenth occurrence of a query term count for much less than the second?

Because relevance saturates: a document mentioning a term fifty times is not fifty times more about it, and without damping, keyword-stuffed documents would dominate the rankings. The k_1 parameter controls how quickly that saturation sets in.

Semantic search solves the synonym problem completely. Why do production systems still run BM25 alongside it?

Because embeddings are weak exactly where lexical search is strong: exact identifiers. An error code, SKU or function name has arbitrary rather than contextual meaning, so it does not embed distinctively and the exact match can rank below topically similar documents. The two fail in opposite directions, which is precisely what makes combining them worthwhile.

Why does Reciprocal Rank Fusion combine ranks rather than scores?

BM25 scores are unbounded and corpus-dependent while cosine similarities sit in a fixed range, so there is no principled way to put them on a common scale, and any normalisation you invent will be fragile across corpora. Ranks are already comparable, so fusing them needs no tuning and no assumptions about either scoring scheme.

A model produces a confident citation to a document that does not exist. Why is this the expected behaviour of the mechanism rather than a malfunction?

The citation is generated by the same next-token prediction as the surrounding text — after 'according to', a plausible-looking reference is simply a high-probability continuation. Nothing in the process consults a source, because there is no index and no lookup step. Grounding has to come from outside the model.

Why can no amount of reranking or query rewriting let a naive RAG pipeline answer 'does the vendor of our payment gateway have SOC 2 certification?'

The second search cannot be formulated until the first one returns the vendor's name, and a linear pipeline retrieves exactly once. Reranking improves the ordering of results for a query you already have; it cannot produce a query that depends on an answer you have not obtained yet. Multi-hop needs a loop.

Why does an agentic retrieval system need an explicit rule for concluding that information is absent?

Because generating another rephrased query is always easier for the model than deciding nothing exists, so without a stopping criterion the agent will keep searching until its budget runs out and then produce something anyway. 'I searched and it is not there' has to be an explicitly permitted and rewarded outcome, backed by a retrieval cap and a no-new-information check.

Where to go next

- Implement BM25 from the formula above over a few thousand of your own documents and compare it head to head against your vector search — the baseline is stronger than most teams assume.
- Add a cross-encoder reranker over the top 50 vector results and measure the change in recall@5; this is usually the largest single accuracy gain available without touching the index.
- Implement RRF over your lexical and semantic retrievers, then find queries where each one rescues the other and use them as regression tests.
- Take ten questions your RAG system answers badly and classify each as a retrieval miss, a multi-hop failure, or a whole-corpus question — the mix tells you which stage of this evolution your system is stuck at.
- Build the conversational query-rewriting step and measure recall on follow-up questions specifically, where naive embedding of pronouns and references fails hardest.
- Instrument an agentic retrieval loop to record how many searches each question triggers, and check the distribution for questions that never terminate cleanly.
- Read the original RAG paper (Lewis et al., 2020) and the ReAct paper (Yao et al., 2022) back to back — the gap between them is exactly the gap between the last two sections of this lesson.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.